Interrogazioni nei sistemi document-based

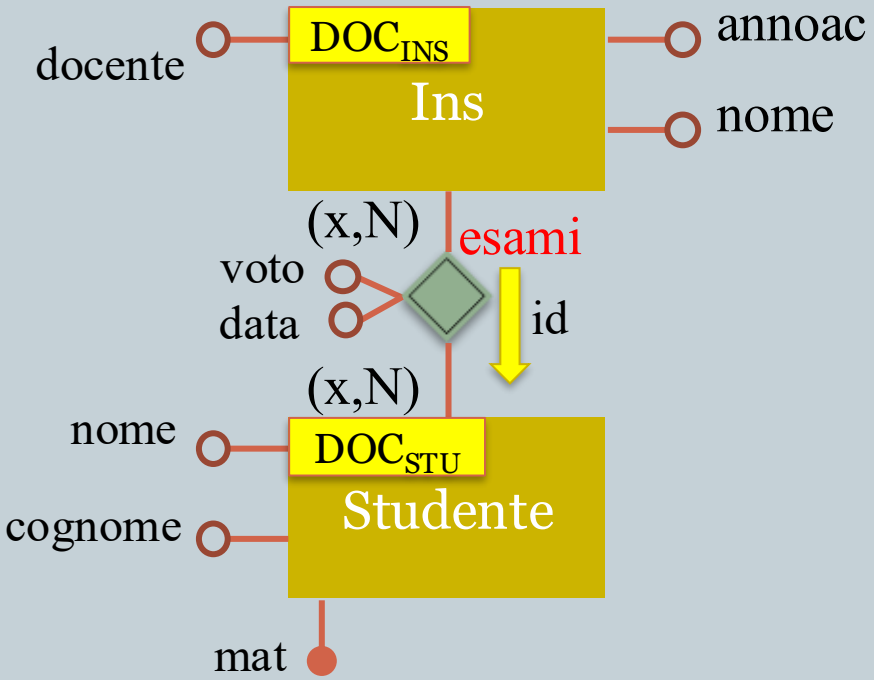
ALBERTO BELUSSI

ANNO ACCADEMICO 2025/2026

Linguaggi di interrogazione nei sistemi NoSQL

- Nei sistemi NoSQL spesso non sono disponibili veri e propri linguaggi di interrogazione come nei sistemi relazionali.
- Solo i sistemi document-based forniscono strumenti simili ad algebra e calcolo relazionale per l'accesso ai dati. Due esempi sono:
  - Couchbase che fornisce il linguaggio **SQL++**, simile all'SQL e dichiarativo;
  - MongoDB che fornisce una **serie di operatori** che consentono di scrivere complesse espressioni algebriche (pipeline) per estrarre dati dalle collezioni di documenti.

Sistemi document-store: esempio



DOC$_{Ins}$

```
{
 _id: autogen
 nome: String
 annoac: String
 docente: String
}
```

DOC$_{Stu}$

```
{
_id: autogen
 nome: String
 cognome: String
 esami: [{Ins_id: integer
          voto: integer
          data: date}
         ,…]
}
```

Sistemi document-store: esempio

Approccio MongoDB

Buona parte del linguaggio di interrogazione di MongoDB è realizzato attraverso il metodo `find`.

Vediamo alcune interrogazioni sull'esempio (Esame incapsulato dentro Studente)

```
db.studenti.insertMany([
{_id: "VR00010", nome: "Mario", cognome: "Rossi"
 esami: [{ins: "Basi di dati", voto: 22, data: "1/7/2016" },
    {ins: "Algebra", voto: 26, data: "5/7/2015" }]}
{_id: "VR00011", nome: "Maria", cognome: "Bianchi"
 esami: [{ins: "Algebra", voto: 30, data: "1/7/2016" }]}
]);
```

Sistemi document-store: interrogazioni

Interrogazioni semplici:

- Trovare tutti gli studenti:

```
db.studenti.find( {} )
```

- Trovare tutti gli studenti di cognome “Rossi”:

```
db.studenti.find( {cognome: “Rossi”} )
```

- Trovare tutti gli studenti di cognome “Rossi” e nome “Mario”:

```
db.studenti.find( {cognome: “Rossi”, nome: “Mario”} )
```

- Trovare tutti gli studenti di cognome “Rossi” o “Bianchi”:

```
db.studenti.find( {cognome:{$in:[“Rossi”,“Bianchi“]}}
)
```

Operatori logici e di confronto

Name	Description
$and	Joins query clauses with a logical **AND** returns all documents that match the conditions of both clauses.
$not	Inverts the effect of a query expression and returns documents that do *not* match the query expression.
$nor	Joins query clauses with a logical **NOR** returns all documents that fail to match both clauses.
$or	Joins query clauses with a logical **OR** returns all documents that match the conditions of either clause.

```
{$or: [{Cognome: "Rossi"},
       {Cognome: "Bianchi"}]}

{$and: [{Cognome: "Rossi"},
        {Nome: "Paolo"}]}
```

```
{$and: [{Esami.voto: {$gte: 21}},
        {Esami.voto: {$lte: 26}}]
```

Name	Description
$eq	Matches values that are equal to a specified value.
$gt	Matches values that are greater than a specified value.
$gte	Matches values that are greater than or equal to a specified value.
$in	Matches any of the values specified in an array.
$lt	Matches values that are less than a specified value.
$lte	Matches values that are less than or equal to a specified value.
$ne	Matches all values that are not equal to a specified value.
$nin	Matches none of the values specified in an array.

Sistemi document-store: interrogazioni

Interrogazioni su dati incapsulati

- Trovare tutti gli studenti che hanno registrato almeno un esame con voto 30:

```
db.studenti.find( {esami.voto: 30} )
```

- Trovare tutti gli studenti che hanno registrato almeno un esame con voto maggiore di 22:

```
db.studenti.find( {esami.voto: { $gt : 22 } } )
```

Sistemi document-store: interrogazioni

Interrogazioni con proiezione

- Trovare il cognome degli studenti di nome "Mario":

```
db.studenti.find( {nome: "Mario"}, {cognome: 1} )
```

- Trovare il cognome degli studenti di nome "Mario" escludendo il campo _id:

```
db.studenti.find( {nome: "Mario"},
                  {cognome: 1, _id: 0} )
```

- Trovare la matricola e i voti di tutti gli studenti:

```
db.studenti.find( { }, {_id: 1, esami.voto: 1} )
```

Sistemi document-store: interrogazioni

Interrogazioni con join (non consigliate dal sistema)

Per eseguire un join tra due collezioni di documenti è necessario usare l'operazione `$lookup` di `aggregate`

Interrogazione sull'esempio 2.3

```
db.STUDENTI.aggregate([
{ $lookup:
        { from: ESAMI,
          localField: _id,
          foreignField: stud_id,
          as: "esami_fatti"
        }
}
])
```

```
STUDENTI: {_id: "VR00010",
 nome: "Mario",
 cognome: "Rossi" }
CORSI: {_id: "ins01",
 nome: "Basi di dati",
 annoac: "2016/2017",
 docente: "Belussi" }
{_id: "ins02",
 nome: "Algebra",
 annoac: "2015/2016",
 docente: "Gregorio"}
ESAMI: {_id: "01",
stud_id: "VR00010"
 ins_id: "ins01",
 voto: 22,
 data: "1/7/2016" }
{_id: "02", stud_id: "VR00010",
 ins_id: "ins02",
 voto: 26,
 data: "5/7/2015" }
```

Sistemi document-store: interrogazioni

Interrogazioni con join

Risultato dell'interrogazione precedente:

```
{_id: "VR00010",
 nome: "Mario",
 cognome: "Rossi",
 "esami_fatti": [
  {_id: "01", stud_id: "VR00010",
   ins_id: "ins01",
   voto: 22,
   data: "1/7/2016" },
  {_id: "02", stud_id: "VR00010",
   ins_id: "ins02",
   voto: 26,
   data: "5/7/2015" } ]
}
```

Sistemi document-store: interrogazioni

Interrogazioni su dati incapsulati con condizioni multiple

- Trovare tutti gli studenti che hanno ottenuto un voto maggiore di 28 nell'appello del 30/10/2025:

```
db.studenti.find(
      { esami: { $elemMatch: { data: '30/10/2025',
                               voto: { $gt : 22 } } } }
```